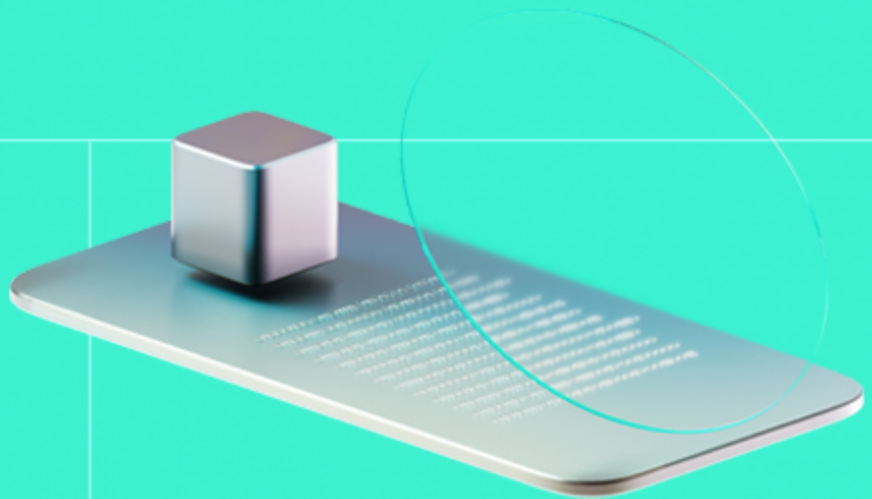




Smart Contract Code Review And Security Analysis Report

Customer: Verasity

Date: 10/04/2026



We express our gratitude to the Verasity team for the collaborative engagement that enabled the execution of this Smart Contract Security Assessment.

Verasity is an open ledger ecosystem bringing trust and transparency to digital advertising and payments.

Document

Name	Smart Contract Code Review and Security Analysis Report for Verasity
Audited By	Khrystyna Tkachuk
Approved By	Kornel Świątłowski
Website	https://verasity.io
Changelog	03/04/2026 - Preliminary Report
	10/04/2026 - Final Report
Platform	Base
Language	Solidity
Tags	Fungible Token; ERC20; Centralization; Vesting
Methodology	https://docs.hacken.io/methodologies/smart-contracts

Review Scope

File Name	contracts.zip
File Hash (SHA256)	8a64bdd
Final File Name	Archive.zip
Final File Hash (SHA256)	191b142

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

7	5	2	0
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	0
Medium	0
Low	0

Vulnerability	Severity Status	
F-2026-15695 - Floating Pragma	Info	Accepted
F-2026-15715 - Redundant Import	Info	Fixed
F-2026-15717 - Use Ownable2Step instead of Ownable	Info	Accepted
F-2026-15719 - Missing Lock Existence Validation in release() Function	Info	Fixed
F-2026-15720 - Misleading NatSpec References Non-Existent revoke() Function	Info	Fixed
F-2026-15722 - Redundant Zero-Address Check in mint() Function	Info	Fixed
F-2026-15723 - Redundant Public Getter Alongside Public State Variable	Info	Fixed

Documentation quality

- Functional requirements are provided.
- Technical description is provided.
 - Run instructions are provided.
 - Technical specification is provided.
- The NatSpec documentation is sufficient.

Code quality

- The code leverages OpenZeppelin contracts and follows established patterns.
- The development environment is configured.

Test coverage

Code coverage of the project is **0%**.

- The project does not include any test cases.

Table of Contents

System Overview	6
Privileged Roles	6
Potential Risks	8
Findings	9
Vulnerability Details	9
Disclaimers	25
Appendix 1. Definitions	26
Severities	26
Potential Risks	26
Appendix 2. Scope	27
Appendix 3. Additional Valuables	28

System Overview

Veracity is an ERC-20 token ecosystem consisting of a feature-rich token contract and an accompanying multi-beneficiary timelock. Together they provide the infrastructure for issuing the PLRL token with centralized compliance controls (minting, pausing, blocklisting) and for distributing tokens to beneficiaries under time-locked distribution.

The token layer is implemented as an abstract base contract **ERC20MintablePausableBlocklistable** that composes OpenZeppelin **AccessControl**, **ERC20Burnable**, and **Pausable**. It adds role-gated minting with a one-way `finishMinting()` switch, a per-account blocklist enforced at the `_update()` level (blocking both sends and receives), and a `transferWithData()` helper that attaches arbitrary calldata to standard transfers via an event. The concrete **PluralToken** contract inherits the base **ERC20MintablePausableBlocklistable** and has the following attributes:

- Name: PLRL
- Symbol: PLRL
- Decimals: 18
- Total supply: 10 000 000 000 tokens.

The distribution layer is handled by **MultiTokenTimelock**, a deposit-model timelock. The admin first sends tokens to the contract via a plain ERC-20 transfer, then calls `addLock()` to allocate deposited tokens to a beneficiary with a specified lock duration. After the lock period elapses, either the beneficiary or the contract owner can call `release()` to transfer the tokens out. Unallocated tokens (deposits that have not been assigned to any lock) can be swept back to any address by the owner via `recoverUnallocated()`. The contract tracks allocations through an auto-incrementing lock ID counter and a `totalAllocated` accumulator that is checked against the contract's actual token balance to derive the unallocated surplus.

Both contracts are non-upgradeable. **PluralToken** uses OpenZeppelin's **AccessControl** for role-based permissioning, while **MultiTokenTimelock** uses OpenZeppelin's **Ownable** for single-owner administration.

Files in scope

- **PluralToken.sol**: ERC-20 token with compliance controls. Defines the abstract **ERC20MintablePausableBlocklistable** base, which layers role-gated minting, global pause/unpause, per-account blocklisting, burning, and a `transferWithData` function that performs a standard transfer and emits an additional event containing caller-supplied `data`.
- **MultiTokenTimelock.sol**: Multi-beneficiary deposit-model token timelock. Accepts a single immutable ERC-20 token address and an initial owner at construction.

Privileged roles

PluralToken.sol (inherits ERC20MintablePausableBlocklistable):

- `DEFAULT_ADMIN_ROLE`: Can grant and revoke all roles (including itself). Serves as the admin role for `MINTER_ROLE`, `PAUSER_ROLE`, and `BLOCK_ROLE`. Assigned to the deployer at construction.

- **MINTER_ROLE**: Can mint an arbitrary amount of tokens to any address via `mint()` (no supply cap enforced). Can permanently and irreversibly disable all future minting via `finishMinting()`. Assigned to the deployer at construction.
- **PAUSER_ROLE**: Can pause and unpause all token transfers globally via `pause()` / `unpause()`. Pausing also prevents minting and burning. Assigned to the deployer at construction.
- **BLOCK_ROLE**: Can add or remove individual addresses from the blocklist via `blockAccount()` / `unlockAccount()`. Blocked addresses cannot send or receive tokens. Assigned to the deployer at construction.

MultiTokenTimelock.sol:

- **owner**: Can create new time-locked token allocations for any beneficiary via `addLock()`. Can release any matured lock on behalf of the beneficiary via `release()`. Can withdraw all unallocated tokens (contract balance minus total locked) to any address via `recoverUnallocated()`. Can transfer ownership to another address in a single step via `transferOwnership()` (inherited from **Ownable**). Set to `initialOwner` at construction.
- **beneficiary** (per-lock, non-privileged): Can call `release()` on their own lock IDs after the lock period has expired. Has no other administrative capabilities.

Potential Risks

- **Centralized Minting to a Single Address:** The `PluralToken` contract concentrates minting tokens in a single address at the deployment, raising the risk of fund mismanagement or theft, especially if key storage security is compromised.
- **Centralized Control of Minting Process:** The `PluralToken` token contract grants the holder of the `MINTER_ROLE` unlimited minting privileges, posing a risk of unauthorized token issuance, potentially diluting the token value and undermining trust in the project's economic governance.
- **Absence of Minting Cap:** The `mint()` function has no supply cap. Any address with `MINTER_ROLE` can mint an arbitrary amount of tokens at any time (until `finishMinting()` is called). If the minter key is compromised, the attacker can inflate the supply indefinitely, destroying token value.
- **Single Points of Failure and Control:** The project is fully or partially centralized, introducing single points of failure and control. This centralization can lead to vulnerabilities in decision-making and operational processes, making the system more susceptible to targeted attacks or manipulation.

Findings

Vulnerability Details

[F-2026-15695](#) - Floating Pragma - Info

Description: In Solidity development, the pragma directive specifies the compiler version to be used, ensuring consistent compilation and reducing the risk of issues caused by version changes. However, using a floating pragma (e.g., `^0.8.20`) introduces uncertainty, as it allows contracts to be compiled with any version within a specified range. This can result in discrepancies between the compiler used in testing and the one used in deployment, increasing the likelihood of vulnerabilities or unexpected behavior due to changes in compiler versions.

The contracts in the codebase currently use floating pragma declarations `^0.8.27`. This increases the risk of deploying with a compiler version different from the one tested, potentially reintroducing known bugs from older versions or causing unexpected behavior with newer versions. These inconsistencies could result in security vulnerabilities, system instability, or financial loss. Locking the pragma version to a specific, tested version is essential to prevent these risks and ensure consistent contract behavior.

Assets:

- PluralToken.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]
- MultiTokenTimelock.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status: Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation:

It is recommended to lock the pragma version to the specific version that was used during development and testing. This ensures that the contract will always be compiled with a known, stable compiler version, preventing unexpected changes in behavior due to compiler updates. For example, instead of using `^0.8.xx`, explicitly define the version with `pragma solidity 0.8.xx;`.

Before selecting a version, review known bugs and vulnerabilities associated with each Solidity compiler release. This can be done by referencing the official Solidity compiler release notes: [Solidity GitHub releases](#) or [Solidity Bugs by Version](#). Choose a compiler version with a good track record for stability and security.

Resolution:

The issue was partially addressed in `191b142`. The **MultiTokenTimelock** pragma version was locked to `0.8.27`, while **PluralToken** still uses the floating pragma version `^0.8.27`.

[F-2026-15715](#) - Redundant Import - Info

Description:

The **MultiTokenTimelock** contract use plain imports of **IERC20** alongside **SafeERC20**:

```
import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

...

using SafeERC20 for IERC20;
```

SafeERC20 already relies on **IERC20**, so the explicit **IERC20** import in **MultiTokenTimelock** appears redundant from a code-maintenance perspective. Keeping both imports increases verbosity and can make the dependency surface look larger than it actually is.

This may lead to reduced code clarity and maintainability, as redundant import can make the contract appear more complex than necessary and may create minor confusion for reviewers or future maintainers.

Assets:

- MultiTokenTimelock.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation:

Remove redundant and unnecessary plain import of **IERC20** to simplify the codebase and improve readability. Consider using named imports from the **SafeERC20** and **Ownable** for consistency:

```
import {IERC20, SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";  
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
```

Resolution:

Fixed in [191b142](#), redundant import statement was removed.

[F-2026-15717](#) - Use Ownable2Step instead of Ownable - Info

Description:

The current implementation of the **MultiTokenTimelock** contract relies on OpenZeppelin's standard **Ownable** contract for managing contract ownership allowing ownership to be transferred in a single step. While functional, this approach poses a potential risk of accidental or unauthorized ownership transfers. For example, the current owner might mistakenly transfer ownership to an incorrect, unintended or inaccessible address. Additionally, once the transfer occurs, the new owner cannot reject or decline the ownership, which can lead to operational challenges.

```
function transferOwnership(address newOwner) public virtual onlyOwner {
    if (newOwner == address(0)) {
        revert OwnableInvalidOwner(address(0));
    }
    _transferOwnership(newOwner);
}

function _transferOwnership(address newOwner) internal virtual {
    address oldOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

Replacing **Ownable** with **Ownable2Step** from OpenZeppelin provides a two-step ownership transfer process, where the new owner must explicitly accept the transfer before it is finalized. This mechanism ensures that ownership transfers are intentional and mutually agreed upon, reducing the risk of accidental or malicious actions.

Assets:

- MultiTokenTimelock.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 3/5

Exploitability: Independent

Complexity: Simple

Severity:

Info

Recommendations**Remediation:**

To improve the security of contract's ownership management, consider implementing the two step ownership transfer by using the **Ownable2Step** contract from OpenZeppelin. This contract implements a two-step process for transferring ownership, requiring the new owner to actively confirm acceptance through a contract call. This mechanism helps prevent the unintended transfer of ownership to an incorrect address, thereby enhancing overall security with an additional verification step.

Resolution:

Accepted, the client has acknowledged the finding and accepted it.

[F-2026-15719](#) - Missing Lock Existence Validation in `release()`

Function - Info

Description:

The `release()` function of **MultiTokenTimelock** contract does not validate whether the provided `lockId` corresponds to an actually created lock. When the caller is the contract owner, all `require` checks pass for a non-existent `lockId` because the default storage values satisfy every condition:

- `lockBeneficiary[lockId]` returns `address(0)` (default for unmapped key).
- The authorization check passes because `msg.sender == owner()` is `true`.
- `locks[lockId]` returns the default `Lock{amount: 0, releaseTime: 0, released: false}`.
- `!lock.released` evaluates to `!false = true` — passes.
- `block.timestamp >= lock.releaseTime` evaluates to `block.timestamp >= 0` — always passes.

```
function release(uint256 lockId) external {
    address beneficiary = lockBeneficiary[lockId];

    require(
        msg.sender == owner() || msg.sender == beneficiary, // @audit msg.sender == owner() - possible that lockId is not exist will revert on line 165 - expective - revert before if lockId is invalid
        "MultiTokenTimelock: not authorized"
    );

    Lock storage lock = locks[lockId];

    require(!lock.released, "MultiTokenTimelock: already released");

    require(
        block.timestamp >= lock.releaseTime,
        "MultiTokenTimelock: lock period not over"
    );

    lock.released = true;
    totalAllocated -= lock.amount; // @audit lock.amount read from storage 3 times

    token.safeTransfer(beneficiary, lock.amount);

    emit TokensReleased(lockId, beneficiary, lock.amount);
}
```

As a result, execution reaches `token.safeTransfer()`. At this point, the call becomes `token.safeTransfer(address(0), 0)`, which eventually triggers OpenZeppelin's ERC-20 `_transfer()` logic and reverts with `ERC20InvalidReceiver(address(0))` because the recipient is the zero address.

The issue results in unnecessary gas consumption and poor error handling for invalid inputs.:

- **Wasted gas:** The transaction traverses all `require` statements, modifies storage (`lock.released = true`), and performs an external call before reverting. The gas consumed up to the revert point is not refunded.
- **Misleading error:** The revert reason is `ERC20InvalidReceiver(address(0))`, which gives no indication that the actual problem is an invalid `lockId`. This makes debugging and off-chain error handling significantly harder for both the owner and frontend integrations.

Assets:

- MultiTokenTimelock.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	3/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Consider adding an explicit existence check at the beginning of `release()`. Since `lockId` is assigned via an auto-incrementing `lockCount` counter, any valid lock must have `lockId < lockCount`:

```
function release(uint256 lockId) external {
    require(lockId < lockCount, "MultiTokenTimelock: lock does not exist");
    address beneficiary = lockBeneficiary[lockId];
    // ... rest of the function
}
```


Resolution:

Fixed in [191b142](#), an explicit lock existence check was added at the beginning of `release()`:

```
function release(uint256 lockId) external {  
    address beneficiary = lockBeneficiary[lockId];  
  
    require(lockId < lockCount, "MultiTokenTimelock: lock does not exist");  
    ...  
}
```

[F-2026-15720](#) - Misleading NatSpec References Non-Existent

revoke() Function - Info

Description:

The NatSpec for `recoverUnallocated()` states "including tokens freed by `revoke()`", implying that a `revoke()` function exists in the contract that can cancel active locks and return their tokens to the unallocated pool. However, no such function is implemented anywhere in **MultiTokenTimelock**. The contract provides no mechanism to revoke, cancel, or modify a lock once it has been created via `addLock()`.

```
/**
 * @notice Withdraw any tokens sitting in the contract that are not
 *         allocated to any lock (including tokens freed by revoke()).
 *
 * @param to Address to send the unallocated tokens to
 */
function recoverUnallocated(address to) external onlyOwner {
    require(to != address(0), "MultiTokenTimelock: zero address");
    uint256 amount = unallocatedBalance();
    require(amount > 0, "MultiTokenTimelock: nothing to recover");

    token.safeTransfer(to, amount);
    emit UnallocatedRecovered(to, amount);
}
```

This leads to:

- **Incorrect trust assumptions:** Users or beneficiaries reading the documentation may believe the owner can revoke their locks at any time, leading to misaligned expectations about the contract's guarantees.
- **Possible missing functionality:** The comment suggests `revoke()` was part of the original design but was either removed or never implemented. If revocation was intended, its absence means the owner has no way to recover tokens from a lock created with incorrect parameters (wrong beneficiary, wrong amount, or excessive duration).

Assets:

- MultiTokenTimelock.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status:

Fixed

Classification

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Consider correcting the outdated comment so that it clearly reflects the implemented logic. Alternatively, if such a function is intended to be part of the design, add it to the contract with proper access control and appropriate lock-status validation.

Resolution: Fixed in [191b142](#), the comment section was corrected to reflect the up-to-date implementation:

```
/**
 * @notice Withdraw any tokens sitting in the contract that are not
 *         allocated to any lock.
 *
 * @param to Address to send the unallocated tokens to
 */
function recoverUnallocated(address to) external onlyOwner {
    require(to != address(0), "MultiTokenTimelock: zero address");
    uint256 amount = unallocatedBalance();
    require(amount > 0, "MultiTokenTimelock: nothing to recover");

    token.safeTransfer(to, amount);
    emit UnallocatedRecovered(to, amount);
}
```

[F-2026-15722](#) - Redundant Zero-Address Check in mint() Function - Info

Description:

The **PluralToken** contract inherits **ERC20MintablePausableBlocklistable**, which provides the `mint()` functionality. This function allows accounts with `MINTER_ROLE` to mint new tokens, provided minting has not been finalized.

The `mint()` function in **ERC20MintablePausableBlocklistable** performs an explicit zero-address validation before calling `_mint()`:

```
function mint(address account, uint256 amount)
    external
    onlyRole(MINTER_ROLE)
{
    require(
        !_mintingFinished,
        "ERC20MintablePausableBlocklistable: minting is finished"
    );
    require(
        account != address(0),
        "ERC20MintablePausableBlocklistable: cannot mint to zero address"
    );

    _mint(account, amount);
}
```

However, this validation is redundant because OpenZeppelin's `ERC20::_mint()` already enforces the same condition internally:

```
function _mint(address account, uint256 value) internal {
    if (account == address(0)) {
        revert ERC20InvalidReceiver(address(0));
    }
    _update(address(0), account, value);
}
```

Both checks validate the same condition, `account != address(0)`, and both revert before any state changes occur. The only difference is the revert mechanism: the wrapper uses a revert string, while OpenZeppelin uses the more gas-efficient `ERC20InvalidReceiver` custom error.

This duplication introduces unnecessary gas overhead on every successful `mint()` call, as the function evaluates an additional condition that is already guaranteed by the underlying ERC20 implementation. It also slightly increases code complexity and creates ambiguity around which

layer is responsible for input validation, making the codebase less clean and harder to maintain over time.

Assets:

- PluralToken.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status:**Fixed****Classification**

Impact Rate:	1/5
Likelihood Rate:	5/5
Exploitability:	Independent
Complexity:	Simple
Severity:	Info

Recommendations

Remediation: Remove the redundant check from `mint()` and rely on OpenZeppelin's built-in validation:

```
function mint(address account, uint256 amount)
    external
    onlyRole(MINTER_ROLE)
{
    require(
        !_mintingFinished,
        "ERC20MintablePausableBlocklistable: minting is finished"
    );
    _mint(account, amount);
}
```

Resolution: Fixed in `191b142`, the redundant custom zero address validation was removed:

```
function mint(
    address account,
    uint256 amount
) external onlyRole(MINTER_ROLE) {
    require(!_mintingFinished, "ERC20MintablePausableBlocklistable: minting is finished");
}
```

```
    _mint(account, amount);  
}
```

[F-2026-15723](#) - Redundant Public Getter Alongside Public State Variable - Info

Description:

The **ERC20MintablePausableBlocklistable** contract declares `_mintingFinished` as a `public` state variable, which causes Solidity to automatically generate a getter named `_mintingFinished()`. At the same time, the contract also defines a custom getter, `isFinishedMinting()`, that returns the same value:

```
bool public _mintingFinished = false;

function isFinishedMinting() public view returns (bool) {
    return _mintingFinished;
}
```

As a result, the contract exposes two externally callable functions with identical behavior in its ABI:

- `_mintingFinished()`: compiler-generated getter
- `isFinishedMinting()`: manually implemented getter

This duplication is unnecessary and increases ABI surface area without adding functional value. It also introduces ambiguity for integrators, frontends, indexers, and other external consumers, which must choose between two equivalent interfaces with no clear indication of which one is intended as canonical.

In addition, exposing `_mintingFinished()` through the public ABI is inconsistent with common Solidity naming conventions, where underscore-prefixed identifiers are typically reserved for internal or private state.

Assets:

- PluralToken.sol
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 5/5

Exploitability: Independent

Complexity: Simple

Severity: Info

Recommendations

Remediation: Consider adopting one of the following approaches:

- Keep the explicit `isFinishedMinting()` getter and make the storage variable `private` or `internal`

```
bool private _mintingFinished;

function isFinishedMinting() public view returns (bool) {
    return _mintingFinished;
}
```

- Keep the state variable public, rename it to a clean external name, and remove the explicit getter:

```
bool public mintingFinished;
```

Resolution: Fixed in `191b142`, the `_mintingFinished` variable was made private, and the public `isFinishedMinting()` function is now the sole getter used to access this value.

Disclaimers

Hacken Disclaimer

The smart contracts given for audit have been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other contract statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited smart contracts.

Appendix 1. Definitions

Severities

When auditing smart contracts, Hacken is using a risk-based approach that considers **Likelihood**, **Impact**, **Exploitability** and **Complexity** metrics to evaluate findings and score severities.

Reference on how risk scoring is done is available through the repository in our Github organization:

[hknio/severity-formula](https://github.com/hacken/severity-formula)

Severity	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to the loss of user funds or contract state manipulation.
High	High vulnerabilities are usually harder to exploit, requiring specific conditions, or have a more limited scope, but can still lead to the loss of user funds or contract state manipulation.
Medium	Medium vulnerabilities are usually limited to state manipulations and, in most cases, cannot lead to asset loss. Contradictions and requirements violations. Major deviations from best practices are also in this category.
Low	Major deviations from best practices or major Gas inefficiency. These issues will not have a significant impact on code execution.

Potential Risks

The "Potential Risks" section identifies issues that are not direct security vulnerabilities but could still affect the project's performance, reliability, or user trust. These risks arise from design choices, architectural decisions, or operational practices that, while not immediately exploitable, may lead to problems under certain conditions. Additionally, potential risks can impact the quality of the audit itself, as they may involve external factors or components beyond the scope of the audit, leading to incomplete assessments or oversight of key areas. This section aims to provide a broader perspective on factors that could affect the project's long-term security, functionality, and the comprehensiveness of the audit findings.

Appendix 2. Scope

The scope of the project includes the following smart contracts from the provided repository:

Scope Details	
File Name	contracts.zip
File Hash (SHA256)	8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716
Final File Name	Archive.zip
Final File Hash (SHA256)	191b14253e95206195f91198feac3aefbdf6bc58d7192c2ab553f3d606d505d8
Whitepaper	N/A
Requirements	Readme.md
Technical Requirements	Readme.md

Asset	Type
MultiTokenTimelock.sol	Smart
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]	Contract
PluralToken.sol	Smart
[8a64bddc94ce781c233ba068c4ba5f1d58c26bc00d1b98772f74e06039d8f716]	Contract

Appendix 3. Additional Valuables

Additional Recommendations

The smart contracts in the scope of this audit could benefit from the introduction of automatic emergency actions for critical activities, such as unauthorized operations like ownership changes or proxy upgrades, as well as unexpected fund manipulations, including large withdrawals or minting events. Adding such mechanisms would enable the protocol to react automatically to unusual activity, ensuring that the contract remains secure and functions as intended.

To improve functionality, these emergency actions could be designed to trigger under specific conditions, such as:

- Detecting changes to ownership or critical permissions.
- Monitoring large or unexpected transactions and minting events.
- Pausing operations when irregularities are identified.

These enhancements would provide an added layer of security, making the contract more robust and better equipped to handle unexpected situations while maintaining smooth operations.

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.